Latest updates: https://dl.acm.org/doi/10.1145/3786228.3786236

RESEARCH-ARTICLE

Exploring Student-AI Interactions in Vibe Coding

FRANCIS GENG, University of California, San Diego, San Diego, CA, United States
ANSHUL SHAH, University of California, San Diego, San Diego, CA, United States
HAOLIN LI, University of California, San Diego, San Diego, CA, United States
NAWAB MULLA, University of California, San Diego, San Diego, CA, United States
STEVE SWANSON, University of California, San Diego, San Diego, CA, United States
GERALD SOOSAI RAJ, University of California, San Diego, San Diego, CA, United States
View all

Open Access Support provided by:
University of California, San Diego
University of Toronto

PDF Download
3786228.3786236.pdf
08 April 2026
Total Citations: 0
Total Downloads: 358

Published: 16 March 2026

Citation in BibTeX format

ACE 2026: 28th Australasian Computing Education Conference
February 9 - 13, 2026
VIC, Melbourne, Australia

ACE '26: Proceedings of the 28th Australasian Computing Education Conference (February 2026)
https://doi.org/10.1145/3786228.3786236
ISBN: 9798400723520

Exploring Student-AI Interactions in Vibe Coding

Francis Geng
University of California, San Diego
La Jolla, CA, USA
fgeng@ucsd.edu

Anshul Shah
University of California, San Diego
La Jolla, CA, USA
ayshah@ucsd.edu

Haolin Li
University of California, San Diego
La Jolla, CA, USA
hal180@ucsd.edu

Nawab Mulla
University of California, San Diego
La Jolla, USA
nmulla@ucsd.edu

Steve Swanson
University of California, San Diego
La Jolla, USA
sjswanson@ucsd.edu

Gerald Soosai Raj
University of California, San Diego
La Jolla, CA, USA
asoosairaj@ucsd.edu

Daniel Zingaro
University of Toronto, Mississauga
Mississauga, Canada
daniel.zingaro@utoronto.ca

Leo Porter
University of California, San Diego
La Jolla, CA, USA
leporter@eng.ucsd.edu

Abstract

Background and Context. Chat-based and inline-coding-based GenAI has already had substantial impact on the CS Education community. The recent introduction of "vibe coding" may further transform how students program, as it introduces a new way for students to create software projects with minimal oversight.

Objectives. The purpose of this study is to understand how students in introductory programming and advanced software engineering classes interact with a vibe coding platform (Replit) to build software and how the interactions differ by programming background.

Methods. Participants were asked to think-aloud while building a web application using Replit. We qualitatively coded screen recordings and associated artifacts (prompts, code changes, and error logs) to construct an interaction labeling scheme capturing how students prompted, tested, debugged, and engaged with code.

Findings. For both groups, the majority of student interactions with Replit were to test common cases in the prototype or use prompts to debug. Only rarely did students analyze or manually edit code. Prompts by advanced software engineering students were much more likely to include relevant app features and codebase contexts than those by introductory programming students.

CCS Concepts

• **Social and professional topics → Computing education**.

Keywords

Large Language Models, Vibe Coding, Novice Programmers, Observation Study

ACM Reference Format:
Francis Geng, Anshul Shah, Haolin Li, Nawab Mulla, Steve Swanson, Gerald Soosai Raj, Daniel Zingaro, and Leo Porter. 2026. Exploring Student-AI Interactions in Vibe Coding. In *28th Australasian Computing Education Conference (ACE 2026), February 09–13, 2026, Melbourne, VIC, Australia.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3786228.3786236

This work is licensed under a Creative Commons Attribution 4.0 International License.
ACE 2026, Melbourne, VIC, Australia
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2352-0/26/02
https://doi.org/10.1145/3786228.3786236

1 Introduction

The advent of generative AI (GenAI) has had considerable impact on computing education. It solves course assignments [6, 8], helps students learn through tutoring [7, 17, 21, 24], alters how we teach [16, 37], and changes the skills we need to teach [18, 27, 38].

LLMs, such as GitHub Copilot and ChatGPT, have changed the workflow of programming for professionals [2] as well as for students [27, 38]. As the CS education research community works to identify the impact of GenAI on student learning (with mixed findings [14, 22, 30, 37]), platforms such as Replit [31], Copilot Agents [23], and Cursor [5] have the potential to alter the landscape even further.

The term "vibe coding" [13, 32, 33] describes a workflow of creating software with minimal oversight—users focus on the end product rather than systematically reviewing or testing code. Existing work in this area has largely examined expert or professional programmers, leaving a gap in understanding how students vibe code. Whether instructors wish students would or would not vibe code, the reality is that these vibe coding tools are available for students to use, and we need to understand the student experience when they do use these tools.

We observed how students from an introductory programming course and an advanced software engineering course used Replit to vibe-code web applications. Participants completed a think-aloud task using Replit's AI Agent and Assistant, and their sessions were qualitatively coded to capture key behaviors including prompting, testing, debugging, and code review.

Our findings suggest two main patterns. First, across both groups, students primarily used Replit to test prototypes and debug through prompts, with limited direct engagement in code. Second, advanced students incorporated more code-level reasoning and richer programming context in their prompts. These differences highlight

that experienced students interact with AI tools in more sophisticated ways, underscoring the importance of teaching students at all levels to read code and frame prompts with appropriate context.

2 Background and Literature Review

2.1 Research on Vibe Coding

In this section, we explore how vibe coding has been defined, summarize the existing research on vibe coding, and compare vibe coding to first-generation coding with GenAI and agentic coding.

2.1.1 What is Vibe Coding? The term "vibe coding" was introduced in February 2025 by Andrej Karpathy, a computer scientist and AI researcher who co-founded OpenAI. The following tweet text is the definition of vibe coding as written by Karpathy [13]:

"There's a new kind of coding I call 'vibe coding', where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g., Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like 'decrease the padding on the sidebar by half' because I'm too lazy to find it. I 'Accept All' always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding — I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works."

That is, vibe coding involves asking a GenAI tool for code, but not reading that code. To debug, one can paste error messages into the AI, or ask for random changes until it's fixed. Karpathy notes that this approach is good enough for weekend projects.

To be clear, Karpathy (an expert computer scientist and AI researcher) doesn't need to vibe code. But, because it works for him, he uses it. It's okay if vibe coding only *"usually"* fixes the bug, or only *"mostly"* works, because Karpathy can just fix the code in these cases. That is, it's perhaps unsurprising that experts, with deep knowledge of the underlying code, can ride the vibes until something goes wrong, fix the problem, and continue vibing. This leaves us with two questions: *what does vibe coding actually look like when carried out by experts?* And, *what happens when novices and intermediate programmers vibe code?* There is initial research into the first question that we describe next; and answering the second question is a goal of the present study.

2.1.2 What Do Expert Developers Do When They Vibe Code? The most comprehensive study of developers vibe coding is the study by Sarkar and Drosos [33]. These authors studied YouTube and Twitch videos of experienced developers who were vibe coding while thinking aloud. The authors chose to analyze those videos where the programmer self-described what they are doing as vibe coding, rather than attempting to impose some criteria of what vibe coding is. That's because the definition of vibe coding is not up to us, but is and will continue to be negotiated by communities of developers who use it in practice.

The authors found that these programmers benefited from their expertise in several ways, such as when evaluating, testing, or manually editing code, and when including detailed technical specifications in their prompts. They are able to rapidly assess code and make immediate judgments about its suitability. The authors use the term "material disengagement" to emphasize the increasing distance between developers and their code in a vibe coding workflow. That is, these developers work on their code not directly, but mediated through GenAI.

The authors' qualitative analysis led to nine top-level categories, including what developers want to build with vibe coding, plans for how they'll build it, the vibe coding workflow, prompting strategies, and debugging. As a sample of what the authors found, we highlight just three of their many insights: 1) Some developers begin with expectations of complete success, but often need to temper those expectations to partial (e.g., 80%) success; 2) Due to its conversational nature, vibe coding often helped these developers go beyond their initial visions. At the same time, vibe coding could be so fast as to lock a developer into a suboptimal plan before they know it; 3) Developers used both single-objective prompts and multi-objective prompts, the latter of which may include directives to the AI about multiple unrelated requirements.

Again, all of these findings are from expert programmers with considerable understanding of programming languages, or the affordances of AI models, or both. In contrast, the novice and intermediate programmers in our study have substantially less experience in software development and AI, making our study a valuable complement to prior work and helping to fill a critical gap in our understanding of vibe coding among non-experts and students.

2.1.3 Vibe Coding vs. Other AI Programming Workflows. While definitions of AI-assisted programming workflows are in flux, we do wish to contrast vibe coding against two other workflows in order to further position vibe coding.

First, we distinguish vibe coding from first-generation GenAI programming workflows from 2022 and 2023, where programmers prompt for each function and the AI completes the code [33]. At this time, chat interfaces hadn't been integrated into the GenAI tools yet. Vibe coding, as we have described, is much further abstracted from the code, allowing programmers to delegate significantly larger tasks to the AI. That said, it is *not* entirely hands off [33].

Second, we distinguish vibe coding from "agentic coding" [32], in which the intention *is* to be hands off. The human is not in the loop: *"agentic coding enables autonomous software development through goal-driven agents capable of planning, executing, testing, and iterating tasks with minimal human intervention"* [32]. At least, this is how agentic coding is defined for experienced developers. We wonder to what extent our comparably less experienced students' "vibe coding" looks like agentic coding.

2.2 How Programmers Interact with AI Programming Tools

Though research related specifically to "vibe coding" is in its early stages, a larger body of work has studied how programmers interact with GenAI programming tools. In this section, we describe the literature related to how experienced and inexperienced programmers interact with AI tools such as GitHub Copilot and ChatGPT.

2.2.1 How Experienced Programmers Use AI Tools. Broadly, the software engineering literature on AI programming tools discusses how these tools can change the software development process [4, 12, 25, 41], how developers use AI tools [2, 9, 35, 36], and challenges associated with using these tools [20, 26, 39, 40]. Here, we focus on how experienced programmers use AI tools. This helps inform how advanced software engineering students (intermediate-level programmers) might interact with AI in a vibe coding platform.

Barke et al. conducted an observational study of 20 programmers with a range of prior experience — from professional to occasional programmers — to theorize how programmers use GitHub Copilot [2]. Their theory posits that programmers interact with Copilot in two modes: exploration mode, in which the programmer uses Copilot to understand how to get started with a task; and acceleration mode, in which programmers are already aware of the necessary steps and use Copilot to speed up implementation [2]. Recent observational studies of how programmers use AI tools support this theory, shedding light on the prompting and verification behaviors among developers [19, 29, 39]. Notably, Liang et al. found that programmers used various techniques to verify AI-generated code output, such as by scanning the code for keywords, using a compiler to detect issues, executing the code, or examining the code in depth [19]. In addition, Vaithilingam et al. and Perry et al. both discussed challenges developers face, with Vaithilingam et al. highlighting the difficulty of comprehending and debugging AI-generated code, and Perry et al. finding that programmers who used AI tools were more likely to write insecure code that included more system vulnerabilities [26, 39]. Other studies focused on how developers process the output of AI tools. For example, Liao and Sundar discussed how users make trust judgments of AI output, engaging in either systematic processing (requiring a careful evaluation of the output) or heuristic processing (involving the use of heuristics to make quick, yet error-prone, trust assessments) [20].

In the computing education research space, Shihab et al. [36] and Shah et al. [34] studied upper-division CS students' interactions with AI tools when working on existing code bases. Shihab et al. conducted a within-subjects study and found that students completed the tasks with Copilot 35% faster, and that Copilot reduced programmers' time spent on writing code by 11% and time spent on performing web searches by 12% [36]. Similarly, Shah et al. analyzed the prompting strategies of 48 students as they completed a task to add a feature to an open-source code base, showing that students preferred to interact with Copilot to comprehend or generate code and reporting a higher trust in Copilot's code comprehension features than its code generation features [34].

The studies mentioned here describe workflows with AI tools where programmers are still actively interacting with the generated code. For example, Barke et al., Liang et al., and Vaithilingam et al. highlight how programmers still spent time reviewing and understanding the code output, especially when trying to debug. However, as discussed in Section 2.1, vibe coding platforms introduce an even greater degree of abstraction between programmers and their code compared to earlier AI tools like Copilot and ChatGPT. This increased separation makes it especially important to study how the more abstract workflow affects intermediate programmers — such as our advanced software engineering students — who have enough experience to understand and occasionally review code, but who may not be proficient or confident in debugging complex implementations by hand.

2.2.2 How Novice Programmers Use AI Tools. Plenty of work has analyzed how introductory students program with AI tools [1, 6, 22, 29, 30], how these tools impact the help-seeking process [7, 11, 17, 24], and how to teach programming in the age of generative AI [8, 17, 18, 27, 38]. Given our study's focus on how novice and intermediate programmers approach vibe coding, we focus here on studies that have observed how novices interact with various AI programming tools.

Novice programmers tend to exhibit ineffective prompting and verification strategies, with multiple studies showing that students write prompts that aim to solve the entire problem at once [1, 15]. Studies have also highlighted novices' ineffective verification strategies working with AI-generated code [15, 29, 30]. Kazemitabaar et al. showed that CS1 students ran the AI-generated code only 60% of the time, and 13% submitted AI-generated code without executing the code at all [15]. These ineffective verification strategies have been observed in non-technical end users as well, who have a tendency to test only minimally before making a judgment about the correctness of AI-generated products [42].

Other studies have aimed to explain why novices struggle when programming with AI. Lucchetti et al. attributed novices' ineffective prompting to difficulty incorporating the context that is needed for the AI to generate effective code; and that, when students get caught in unproductive prompting cycles, it's often because they continue making surface-level wording changes rather than adding what's missing [22]. Prather et al. and Tankelevitch et al. argued that some of the challenges are metacognitive [30, 37]. For example, novice students struggle to verify the correctness of output, leading them down an incorrect path that may only compound their existing struggles [30].

In short, the studies above focused on how novices use and struggle with first-generation AI tools that involve more programmer-code interactions than vibe coding. In response, our study takes the first steps in understanding how novice programmers prompt and verify the output of a vibe coding platform that, by design, lowers the interactions between programmers and code.

3 Methods

3.1 Research Questions

The research questions for our study are as follows:

RQ1 How do students interact with AI tools when engaging in a vibe coding workflow?

RQ2 How does prior programming experience influence the way students develop software in a vibe coding workflow?

3.2 Study Context

3.2.1 Recruitment. We recruited students from two computer science courses at a large research-intensive North American university, under an approved Human Subjects protocol. The first course (CS1) primarily enrolls first- and second-year students with some prior programming experience. This 10-week Java course covers variables, conditionals, arrays, recursion, and basic object-oriented programming. The second course, an advanced Software

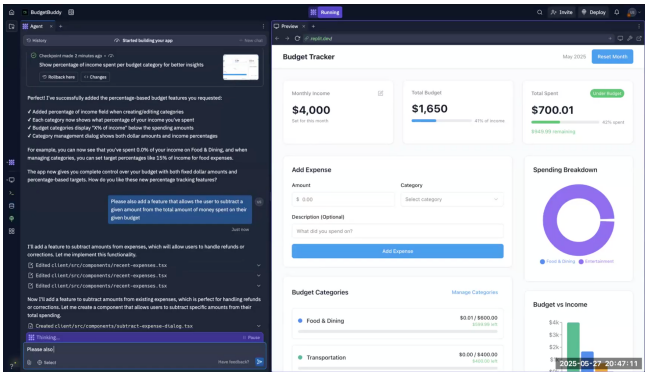

Figure 1: Screenshot of the Replit user interface during a task.

Engineering (SWE) class, emphasizes understanding, managing, and contributing to legacy codebases. It primarily serves final-year students who have completed the core software engineering course, where they learn AGILE development and build mobile or web applications from scratch in a team setting.

3.2.2 Replit. Replit is a browser-based, AI-integrated IDE designed to support a vibe coding workflow [31]. It invites users to *"Prompt your app ideas to life — no coding required"*[1]. The interface combines an AI chat panel to interact with chatbots and a live preview of the application interface (Figure 1). Initially static, the preview becomes interactive as it reflects the evolving application (*"iterative prototype"*). Unlike traditional IDEs such as VS Code or Eclipse, Replit hides code by default, centering development on AI interaction and prototype feedback rather than manual code editing.

As of May 2025 (when the study was conducted), Replit's "AI App Builder" scaffolded full-stack web applications primarily in TypeScript, unless users requested another framework. The platform automatically handled environment setup, file organization, dependencies, and deployment, allowing students to run and interact with a live application within minutes of their first prompt.

Within this interface, Replit offers two distinct chatbot tools, "Replit Agent" and "Replit Assistant". The Agent is optimized for building applications from the ground up; it can autonomously scaffold full-stack projects, configure environments, and directly create or modify files based on high-level prompts. In contrast, the Assistant is tailored for working within an existing project. It supports code-level refinement tasks such as debugging, explaining code behavior, refactoring, and minor feature edits. It functions similarly to an in-line coding collaborator: users can ask questions or request specific changes, and the Assistant proposes contextual edits that must be explicitly approved before being applied.

3.3 Participants

We recruited 9 students from the CS1 course and 10 students from the SWE course (see Section 3.2.1 for descriptions of these courses).

Within the CS1 cohort (CS1_1–9), 4 students were majoring in computing or related fields and 5 were not. 6 identified as men and 3 as women; 7 identified as Asian or Asian American and 2 as White. 6 students reported confidence in building small class projects, while 3 had only basic programming skills. None had software engineering internship experience involving legacy codebases.

All 10 SWE participants (SWE_1–10) were computing majors, with 8 men and 2 women; 7 identified as Asian or Asian American, 2 as Chicanx or Latinx, and 1 as White. 4 reported being comfortable with small projects, 4 with intermediate projects, and 2 with large projects or legacy codebases. 6 had prior internships, and 9 aspired to become professional software engineers.

3.4 Study Procedure

The study sessions were conducted one-on-one on Zoom by the first author. To refine task clarity, we piloted the procedure with six students from the same institution—two from the CS1 course and four upper-division undergraduates with backgrounds similar to the SWE participants. The finalized two-hour study comprised three phases:

Pre-task training (30 minutes): Students watched a six-minute tutorial demonstrating the use of Replit Agent and Assistant, accessing the codebase, and interacting with the prototype. The researcher then introduced the think-aloud method via a short live demo, after which students practiced the procedure on a sample task using the Replit platform.

Development session (60 minutes, timed): Students were asked to create a personal budget management web app in Replit. The task required four core functions: (1) set a monthly budget; (2) break it into categories (e.g., food, rent, entertainment); (3) record expenses; and (4) compare spending to goals. We made the task description open-ended to encourage diverse student interpretations and resulting application designs. Students could use any Replit features (Agent, Assistant, code editor, console, or preview) and external tools such as ChatGPT.

The timer began when students opened the task description. Sessions followed the think-aloud protocol, with recordings capturing voice and screen actions; cameras were optional. The researcher observed and took notes throughout.

Post-task interview (30 minutes): Each participant completed a short semi-structured interview covering background, initial impressions of Replit, challenges, prior development experience, confidence, and reflection. Example questions included:

- What was it like conveying your goals to Replit?
- I noticed that you accessed (or did not access) code during the task — could you share your reasoning?
- Did your prior programming experience influence how you dealt with challenges?
- If you were to re-do the entire activity, what would you do differently from your first attempt?

These questions are not exhaustive but are meant to illustrate the kinds of probes used to support our interpretation of observed behaviors and contextualize the student quotes that we report. We did not perform a full thematic analysis of the interviews or think-aloud data for this paper; instead, we selectively drew on them to provide context. Accordingly, the analysis reported in this paper focuses on a reproducible labeling of observable interaction events in the IDE (and associated artifacts), rather than a reflexive thematic analysis of participants' verbal accounts. A subsequent paper will present deeper qualitative analysis.

[1] https://replit.com/usecases/ai-app-builder

Table 1: Labeling scheme based on student-AI interactions, divided into four major categories.

Label	Definition
Interacting with Prototype	
Refresh Prototype	Reloading the tab that displays the prototype
Test Common Case	Interacting with the prototype using typical inputs or actions that reflect normal usage; repeated clicks or filling out a form counts as a single case
Test Edge Case	Interacting with the prototype using unusual, invalid, or boundary inputs such as negative numbers and empty fields
Writing a Prompt	
Debug	Prompting to address an error, bug, or system failure
Add/Remove/Update Core Feature	Prompting to request changes to a core feature (e.g., adding, checking, deleting, or editing budgets, budget categories, or expenses) without an error or bug
Add/Remove/Update Non-core Feature	Prompting to request changes to a non-core feature (e.g., UI elements, spending graphs) without an error or bug
Ask Clarification Question	Prompting to ask the AI to explain, define, or elaborate on a concept, term, feature, or technical details
Brainstorm Ideas	Prompting for open-ended ideas or approaches without a clearly defined solution
Other	Prompting actions not covered by other labels, such as simulating a test case or responding to the AI's questions
Managing Replit Workflow	
Accept Code Change	Accepting a code modification proposed by the Replit Assistant using the built-in apply-suggestion feature
Pause AI During Generation	Interrupting the AI's response before completion by clicking the Replit pause or stop button
Resume AI Generation After Pause	Resuming a paused AI response by clicking the Replit continue or resume button
Revert to Checkpoint	Reverting the code to a previous version by selecting a checkpoint created by Replit
Load Preview from Checkpoint	Loading a preview of code from a previous checkpoint without reverting to it
Approve Plan with X Additional Features	Approving 0 or more optional features from the Replit Agent's initial project plan
Engaging with Code/log	
Interpret	Spending significant time analyzing code, console logs, or other development-related outputs
Edit	Modifying code, console logs, or other development-related outputs

3.5 Data Analysis

Following established guidance on inductive qualitative analysis, we used an iterative coding process to identify recurring interaction patterns [3]. As we did not conduct a full thematic analysis of think-aloud and interview data, the labels reported here should be interpreted as behavioral interaction categories derived primarily from screen-recorded activity.

Each session was coded as an ordered sequence of interaction events. An event begins when the participant initiates a distinct action in the workflow and ends when they transition to a different action category. Events were defined by observable boundaries rather than fixed time windows. Repeated micro-actions within the same attempt (e.g., multiple clicks while testing the same flow) were treated as a single event.

Two authors jointly open-coded one CS1 and one SWE session to generate an initial codebook and decision rules. Through iterative discussion, we consolidated these into four top-level categories with operational definitions for each label. The two authors then independently coded the remaining 17 sessions and met periodically to calibrate interpretations and refine decision rules. Finally, disagreements were resolved through negotiated agreement [10, 33] by re-watching the relevant segments and referencing associated artifacts to apply the current decision rules. The finalized labeling scheme is shown in Table 1.

We additionally recorded which AI tool students used for *Writing a Prompt* (Replit Agent, Replit Assistant, ChatGPT) and created sublabels for the *Writing a Prompt—Debug* category (Table 2), which represented most prompting interactions and the greatest variation in communication styles. Other prompting categories were less frequent and we did not further subdivide them.

4 Results

4.1 RQ1: Student-AI Interactions in Vibe Coding

Overview. The overall interactions with AI tools across the 19 participants are shown in Figure 2. Across all students, the most prevalent label was *Interacting with Prototype* (63.61% of all labels, n = 1164), followed by *Writing a Prompt* (20.60%, n = 377), *Managing Replit Workflow* (8.42%, n = 154), and *Engaging with Code/log* (7.38%, n = 135). However, differences emerged across cohorts: students in CS1 showed markedly higher proportions of *Writing a Prompt* labels compared to SWE students, whereas *Engaging with Code/log* interactions were more common among SWE students.

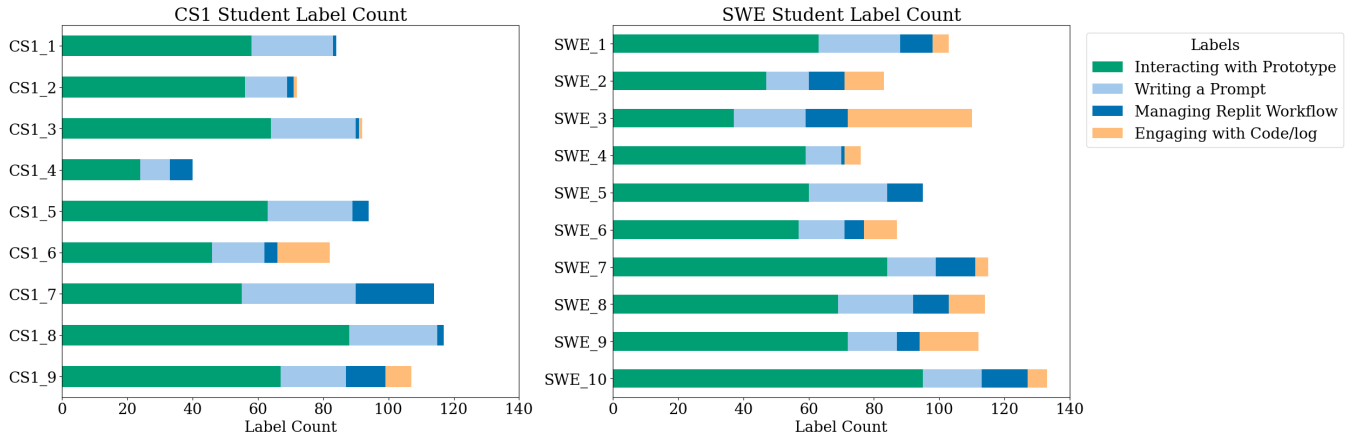

Figure 2: Distribution of labels across all 19 students, grouped by course. Each bar represents a single student, segmented by interaction categories: *Interacting with Prototype, Writing a Prompt, Managing Replit Workflow,* and *Engaging with Code/log.* SWE_1 to SWE_10 are SWE students and CS1_1 to CS1_9 are CS1 students.

Table 2: *Writing a Prompt — Debug* Sublabeling Scheme.

Sublabel	Definition
Error Message	Including a partial or full system error message when describing a bug or issue
Failing Case	Describing the input, condition, or steps that led to a failure or error, including what preceded the problem
Code-focused	Referencing specific files, code snippets, or implementation details believed to be related to the issue
Other Details	Providing other relevant information not captured above, such as triggering AI-generated debugging prompts (e.g., *"Troubleshoot this issue"* or *"Ask Agent to explain code"*)
Low Context	Debugging prompts with none of the above sublabels selected, showing little or no new information or direction when initiating or continuing a debugging effort

Restarters. While prompting was a common behavior across participants, a subset of students (4 out of 19) exhibited a rather dramatic prompting pattern: they restarted the entire project using the Replit Agent mid-task. We term these students *restarters* (SWE_1, SWE_2, CS1_4, CS1_7). 3 out of the 4 *restarters* restarted primarily to simplify their interaction with the AI, citing overwhelming or ambiguous behavior from earlier prompts. For instance, one student reflected that they *"asked the Replit to do way too many things,"*

making it difficult to identify specific issues (SWE_1, interview). Another opted to *"break it down one task at a time"* after experiencing repeated failures (CS1_7, think aloud). These behaviors suggest that some students use restart strategies not out of failure alone, but as a form of iterative refinement and task decomposition. The fourth *restarter* (SWE_2) retained their original prompt but added a sentence to explicitly request a Flask framework over React due to familiarity and perceived simplicity, and they considered their restart as a means of architectural realignment rather than functional simplification. This finding highlights a critical point: restart decisions reflected metacognitive awareness about the limitations of debugging through prompting and demonstrated students' adaptive strategies when existing prototypes became unmanageable.

Table 3: Frequency of labels for *Interacting with Prototype* actions.

Label	Count	Percent
Test Common Case	1067	91.67%
Refresh Prototype	71	6.10%
Test Edge Case	26	2.23%
Total	1164	100%

Interacting with Prototype. We examine how students interact with the prototypes generated by the AI tools (Table 3). The overwhelming majority of these interactions (91.67%) involved testing common use cases, while only 6.10% involved refreshing the prototype and a mere 2.23% involved edge case testing. Transcript data suggests that prototype refreshes were typically prompted by technical limitations (e.g., Replit failing to maintain state across pages) rather than as part of a deliberate debugging strategy. Notably, no student wrote or executed unit tests during the study, indicating that their approach to testing was exclusively centered on feature-level behaviors visible through the UI. The absence of structured test practices, combined with minimal edge case coverage, suggests that many students' testing strategies remained limited in scope and were shaped by immediate usability and tool constraints. These patterns highlight the inherently iterative and sometimes unstable nature of vibe coding, where students often remain occupied with basic interactions and repeated troubleshooting, rather than advancing toward comprehensive feature validation.

Table 4: Frequency of labels for *Writing a Prompt* actions.

Label	Count	Percent
Debug	230	61.01%
Add/Remove/Update Non-core Feature	63	16.71%
Add/Remove/Update Core Feature	53	14.06%
Other	16	4.24%
Ask Clarification Question	12	3.18%
Brainstorm Ideas	3	0.80%
Total	377	100%

Writing a Prompt. To better understand students' goals when they are prompting, we analyzed the labels of all prompting behaviors for all 19 students (Table 4). The majority of prompts (61.01%) were used for debugging AI-generated code, followed by modifications to non-core features (16.71%) and core features (14.06%). Prompts related to brainstorming, clarification questions, or miscellaneous tasks were rare (less than 5% each). These data indicate that students primarily engaged with AI tools to troubleshoot and refine partial implementations, rather than to build core functionality from scratch. As one student described their strategy: *"...finding [bugs] myself, realizing what the problem was, and then putting it back into the Agent to solve it in like 2 seconds..."* (SWE_9, interview). This reflects how students often used AI to efficiently resolve implementation issues they had manually identified.

Table 5: Frequency of student prompts with different AI tools.

AI Tool	Count	Percent
Replit Assistant	271	50.94%
Replit Agent	246	46.24%
ChatGPT	15	2.82%
Total	532	100%

In terms of which AI tools students used for prompting (Table 5), the Replit Assistant accounted for a slight majority of prompting interactions (50.94%), closely followed by Replit Agent (46.24%). ChatGPT was used only in 2.82% of the prompt instances, likely reflecting its auxiliary role in the workflow. While multiple AI tools were available, most of the prompting occurred within the embedded Replit interfaces, suggesting that accessibility and immediacy of tooling may play a significant role in shaping student behavior. As one student reflected, *"I didn't really understand the real difference between Agent and Assistant... it kind of felt like they were the same thing"* (SWE_4, interview), emphasizing how conceptual ambiguity may have contributed to balanced usage. Another student explained, *"I'll kind of use ChatGPT and Copilot for assistance on making small fixes,"* (SWE_10, interview) illustrating ChatGPT's occasional and supporting role relative to the Replit-native tools.

Managing Replit Workflow. In addition to prompting actions, students interacted with Replit in several other ways that reflect the vibe coding workflow (Table 6). The most common action in this category was accepting Replit-proposed code changes (68.18%), a manual decision-making step mandatory for students working with Replit Assistant. After writing an initial prompt, a student had to approve an AI-generated implementation plan with optional additional features recommended by the Replit Agent (14.94%) based on the content of that first prompt. Some students also paused (7.79%) and resumed (1.30%) the AI generation process, indicating moments of review or reconsideration during code generation. In addition, a few students used Replit's version control features to either *Revert to Checkpoint* (4.55%) or *Load Preview from Checkpoint* (3.25%), actions functionally similar to Git's `revert` and `checkout`, respectively. These behaviors suggest that students occasionally chose to backtrack when a prompt yielded undesirable or destabilizing changes, emphasizing the trial-and-error nature of vibe coding.

Table 6: Frequency of labels for *Managing Replit Workflow* actions.

Label	Count	Percent
Accept Code Change	105	68.18%
Approve Plan with X Additional Features	23	14.94%
Pause AI During Generation	12	7.79%
Revert to Checkpoint	7	4.55%
Load Preview from Checkpoint	5	3.25%
Resume AI Generation After Pause	2	1.30%
Total	154	100%

Engaging with Code/log. Among the rare cases where students engaged with AI-generated code and logs, 90.37% of the interactions were interpreting and reading code (n = 122), and the remaining actions were direct edits (9.63%, n = 13). This strong preference for interpretation over modification suggests that students were often hesitant to alter AI-generated code, likely due to limited familiarity with the underlying implementation. The vibe coding workflow may exacerbate this by distancing students from the logic and structure of AI-generated codebases. As one student explained, *"Because so much of it was just done by the LLM, I had a lesser understanding of the codebase — rather than what I would do on my own, where I know what each line does"* (SWE_6, interview).

4.2 RQ2: Relationship between Programming Experience and Vibe Coding Strategies

All statistical comparisons in this section use two-sample independent t-tests (df = 17).

Interacting with Prototype. We analyzed the extent to which students conducted edge case testing while *Interacting with Prototype* using uncommon or boundary inputs. 6 of 10 SWE students performed at least one edge case test, compared to 4 of 9 in CS1. While this difference is modest, it may reflect task progression: students who encountered repeated errors in basic functionality often focused on common case testing and did not reach a point where testing edge cases was necessary. Since edge case testing is introduced early in many computing curricula, including this CS1 course, the observed frequency difference between cohorts is less likely due to instruction and more likely tied to whether the student's implementation had stabilized enough to allow for more nuanced evaluation.

Writing a Prompt. Students from CS1 and SWE showed noticeably different behaviors in two aspects when using debugging

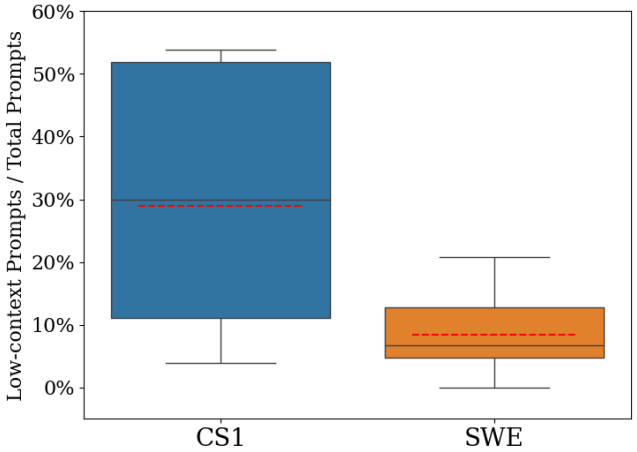

Figure 3: Proportion of *Low Context* prompts per student, normalized by each student's total number of prompts.

prompts during vibe coding, characterized as *Low Context* and *Code-focused* (defined in Table 2). *Low Context* prompts are often vague and offer limited actionable context, such as *"The buttons appear but they don't work at all,"* or follow-up phrases like *"it's still not working."* Figure 3 compares the proportion of low context prompts across the two groups and reveals that CS1 students had a significantly higher proportion of such prompts (M = 28.89%, SD = 20.66%) than SWE students (M = 8.39%, SD = 6.99%, p <.01).

In contrast, *Code-focused* prompts that reference specific segments of AI-generated code or logs were more prevalent among SWE students (5.83% of SWE students' prompts) than CS1 students (0.60% of CS1 students' prompts). For example, one student wrote, *"In line 118 of routes.ts, are you passing the correct data into storage.createStorage?"* (CS1_6, prompt). Another student wrote, *"In the <Header> component, error occurred 'cannot read properties of undefined (reading 'month')', set the month to be the current month at the time the user is using the web app"* (SWE_8, prompt).

These kinds of prompts demonstrate precise understanding and reasoning about the program's structure and behavior. 7 out of 10 SWE students used code-focused prompts, while only 1 student from the CS1 group (CS1_6) did so. This difference suggests that students with greater programming experience were more capable of constructing prompts that referenced concrete implementation details, allowing them to more effectively communicate intent and troubleshoot with the AI system.

Engaging with Code/log. To investigate how programming experience influences students' engagement with code, we compared the proportion of *Engaging with Code/log* actions (either interpreting or editting AI-generated code) between students in CS1 and SWE. As shown in Figure 4, SWE students exhibited a higher overall proportion of *Engaging with Code/log* behaviors (M = 10.56%, SD = 9.83%) compared to CS1 students (M = 3.27%, SD = 6.55%, p = .078). Among individual behaviors, 9 of 10 SWE students engaged in code interpretation, and 3 of them also edited AI-generated code. By contrast, 4 of 9 CS1 students interpreted code, only 2 of whom (CS1_6 and CS1_9) edited code. Notably, these two students were the only CS1 students who self-identified as capable of developing intermediate-level programs and debugging, suggesting that

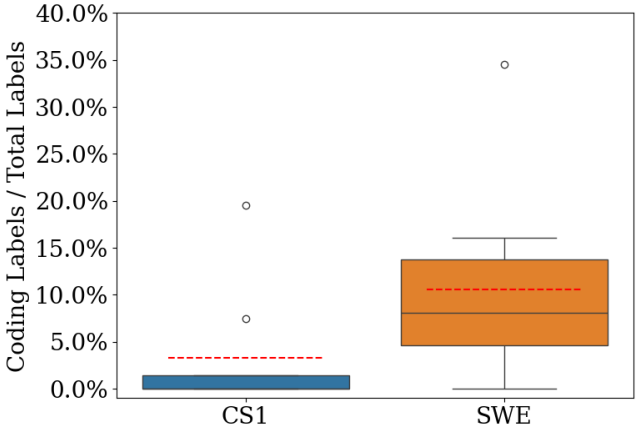

Figure 4: Proportion of *Engaging with Code/log* labels per student, normalized by each student's total behavior labels.

students with greater programming experience are more likely to engage directly with AI-generated code.

5 Discussion

With the recent rise of agentic AI and vibe coding, there is considerable discourse around higher-level prompting as a viable practice for creating software. That discourse, however, has been focused mostly on professional software engineers, not students. The fact that vibe coding involves minimal supervision may suggest that someone completely unfamiliar with programming could just dive right into the practice. Our findings raise caution here, in that students from both introductory and advanced programming classes relied on their existing programming skills when interacting with AI: prompting to debug, performing feature-level tests, and engaging with code. This suggests that someone with no or very limited programming background may struggle with vibe coding, since even our CS1 students drew on foundational skills to make progress.

5.1 Implications for Educators

In our findings, interacting with the prototype was the most common behavior for both introductory and advanced students. Both groups experimented with the prototype to identify what worked and what didn't, then used that information to ask Replit to correct shortcomings. The fact that students were so quick to move to testing is an encouraging finding for teaching testing in computing courses. However, it is critical to understand the type of testing students engaged in and its implications. Students in our study primarily performed closed box testing—behavioral testing focused on whether features work correctly, without examining or understanding the underlying code. The Replit interface reinforces this approach by intentionally masking code behind one to two button clicks, presenting only the chat prompts and prototype preview by default. While students technically had access to the generated code, most intentionally chose not to engage with it. This stands in contrast to open box testing, such as unit testing, where developers have full access to and understanding of the code structure and can validate that individual functions or components work correctly in isolation. While closed box testing develops valuable skills in

validating user-facing functionality, it does not build the deeper understanding required for maintaining, debugging, or extending code at scale. Students may graduate competent at verifying that generated code produces correct outputs for specific inputs, yet struggle when asked to integrate their solutions into larger codebases or optimize performance. Educators should explicitly teach students the differences between these testing approaches and when each is appropriate. Furthermore, since Replit and similar tools encourage feature-based testing over unit tests, instructors may need to deliberately scaffold assignments that require students to write unit tests and engage with code structure.

Our findings also demonstrate that introductory and advanced programming students have different prompting styles. The advanced students are far more apt to communicate with Replit using prompts that contain computational ideas and details than introductory students. These sophisticated prompts suggest, perhaps unsurprisingly, that advanced students drew on more computational ideas when interacting with Replit. While we do not measure prompt success in this study, the difference in style highlights that prompting in vibe coding involves not just phrasing requests, but conveying intent grounded in code structure, logic, and context. This finding supports recent work by Lucchetti et al., who found that programming novices often author prompts lacking important technical details necessary for AI to generate appropriate solutions [22]. The pedagogical implication is clear: *prompting effectiveness depends fundamentally on programming knowledge*. Students are unlikely to be able to effectively direct AI to generate appropriate code without understanding what "appropriate" means computationally. They may struggle to debug AI-generated errors if they are unaware of the kinds of errors that are possible. They cannot iteratively refine prompts without understanding why initial attempts failed. We argue that computing education should not reduce emphasis on foundational programming concepts in favor of prompting instruction, but rather recognize that strong prompting emerges from strong programming foundations.

5.2 Limitations

Sample representation. Our study was conducted at a single North American institution with students recruited from two computing courses, and participation was voluntary. This recruitment strategy may introduce self-selection bias, as students who opted into the study may be more confident, motivated, or interested in AI tools than their peers. Consequently, the behaviors observed may reflect a more engaged or higher-performing subset of the student population. Expanding future studies to multiple institutions and course settings would strengthen the applicability of findings across contexts.

Platform and language specificity. Our findings are grounded entirely in student experiences with the Replit platform, which consists of an AI chat interface and an interactive prototype that deprioritizes code visibility. While this design makes Replit ideal for examining vibe coding, it does not reflect the full spectrum of vibe coding platforms. Other tools (e.g., Cursor, Copilot Agents) may provide greater transparency into generated code, tighter integration with traditional IDE workflows, or different prompting paradigms. We also note that TypeScript, the programming language favored by Replit, was unfamiliar to many of our CS1 participants, perhaps interfering with their interest or capacity in working with the generated code. Future work could incorporate other vibe coding platforms and programming languages to assess how specific design features and language familiarity shape student behavior.

Focus on observed behaviors, not underlying reasoning. Our analysis centers on students' observable actions (e.g., prompting, testing, engaging with code) and classifies them using a structured labeling scheme. However, we did not systematically analyze students' spoken reasoning or reflective interview data to explain *why* those behaviors occurred. While the think-aloud protocol and post-task interviews provide valuable context, we primarily used them to validate labeled behaviors rather than drive deeper thematic or cognitive analysis. As a result, we cannot fully characterize students' intentions, mental models, metacognitive strategies, or emotional responses. A complementary analysis of verbal data could provide richer insight into students' goals, frustrations, and strategies while vibe coding.

Interaction frequency is not equivalent to time on task. All quantitative results in this paper are based on counts of interaction labels (e.g., number of prompts, code edits, test cases). However, raw frequency does not capture *how long* students spent on different activities. For instance, a single instance of code interpretation may involve several minutes of analysis, while five quick prompts could span less than a minute in total. As such, interpreting our frequency data as proportional effort or cognitive investment would be misleading. Future work should incorporate duration-based metrics or temporal coding to accurately represent time allocation and sustained engagement with specific activities.

6 Conclusion

This study offers first insights into how computing students interact with a vibe-coding tool (Replit) while creating a functional and useful web app. Through qualitatively analyzing observations of how students vibe code, we uncovered students' interaction patterns with Replit and behavioral differences between students in introductory and advanced computer science courses. Our findings include 1) the bulk of student interactions with Replit are closed box testing and prompting to debug, and 2) advanced students are more capable of prompting with computational sophistication and are more likely to interact with the codebase. Our findings offer insights to educators on the skills that students use when vibe coding while buttressing findings in GenAI broadly — that testing and debugging remain critical in the AI era [27, 28] and that students need training in prompting [22].

Acknowledgments

This work is supported by the NSF Graduate Research Fellowship Program and NSF Award #2417374, and by the Google Award for Inclusion Research Program. Any opinion, findings, and conclusions or recommendations expressed in this material are those of the authors(s) and do not necessarily reflect the views of the NSF. ChatGPT was used for light language editing, and all content was reviewed, verified and revised by the authors.

References

[1] Matin Amoozadeh, Daye Nam, Daniel Prol, Ali Alfageeh, James Prather, Michael Hilton, Sruti Srinivasa Ragavan, and Amin Alipour. 2024. Student-AI interaction:

A case study of CS1 students. In *Proceedings of the 24th Koli Calling International Conference on Computing Education Research*. Article 13. https://doi.org/10.1145/3699538.3699567

[2] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with code-generating models. *Proc. ACM Program. Lang.* 7, OOPSLA1 (2023), 27 pages. https://doi.org/10.1145/3586030

[3] V. Braun and V. Clarke. 2021. *Thematic Analysis: A Practical Guide*. SAGE Publications.

[4] Xiang Chen, Chaoyang Gao, Chunyang Chen, Guangbei Zhang, and Yong Liu. 2025. An Empirical study on challenges for LLM application developers. *ACM Trans. Softw. Eng. Methodol.* (2025). https://doi.org/10.1145/3715007

[5] Cursor. 2025. Cursor - The AI Code Editor. https://cursor.com/en.

[6] Paul Denny, Viraj Kumar, and Nasser Giacaman. 2023. Conversing with Copilot: Exploring prompt engineering for solving CS1 problems using natural language. In *Proceedings of the 54th ACM technical symposium on computer science education V. 1*. 1136–1142.

[7] Paul Denny, Stephen MacNeil, Jaromir Savelka, Leo Porter, and Andrew Luxton-Reilly. 2024. Desirable characteristics for AI teaching assistants in programming education. In *Proceedings of the 2024 conference on Innovation and Technology in Computer Science Education V. 1*. 408–414.

[8] Paul Denny, James Prather, Brett A Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N Reeves, Eddie Antonio Santos, and Sami Sarsa. 2024. Computing education in the era of generative AI. *Commun. ACM* 67, 2 (2024), 56–67.

[9] Ya Gao. 2024. Research: Quantifying GitHub Copilot's impact in the enterprise with Accenture. https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-in-the-enterprise-with-accenture/

[10] D.R. Garrison, M. Cleveland-Innes, Marguerite Koole, and James Kappelman. 2006. Revisiting methodological issues in transcript analysis: Negotiated coding and reliability. *The Internet and Higher Education* 9, 1 (2006), 1–8. https://doi.org/10.1016/j.iheduc.2005.11.001

[11] Irene Hou, Owen Man, Kate Hamilton, Srishty Muthusekaran, Jeffin Johnykutty, Leili Zadeh, and Stephen MacNeil. 2025. "All roads lead to ChatGPT": How Generative AI is eroding social interactions and student learning communities. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1*. 79–85. https://doi.org/10.1145/3724363.3729024

[12] Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, and Huaming Chen. 2025. From LLMs to LLM-based agents for software engineering: A survey of current, challenges and future. arXiv:2408.02479 [cs.SE] https://arxiv.org/abs/2408.02479

[13] Andrej Karpathy. 2025. https://x.com/karpathy/status/1886192184808149383

[14] Majeed Kazemitabaar, Justin Chow, Carl Ka To Ma, Barbara J Ericson, David Weintrop, and Tovi Grossman. 2023. Studying the effect of AI code generators on supporting novice learners in introductory programming. In *Proceedings of the 2023 CHI conference on human factors in computing systems*. 1–23.

[15] Majeed Kazemitabaar, Xinying Hou, Austin Henley, Barbara Jane Ericson, David Weintrop, and Tovi Grossman. 2024. How novices use LLM-based code generators to solve CS1 coding tasks in a self-paced learning environment. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research*. Article 3, 12 pages. https://doi.org/10.1145/3631802.3631806

[16] Majeed Kazemitabaar, Oliver Huang, Sangho Suh, Austin Henley, and Tovi Grossman. 2025. Exploring the design space of cognitive engagement techniques with AI-generated code for enhanced learning. In *Proceedings of the 30th International Conference on Intelligent User Interfaces*. 695–714.

[17] Majeed Kazemitabaar, Runlong Ye, Xiaoning Wang, Austin Henley, Paul Denny, Michelle Craig, and Tovi Grossman. 2024. CodeAid: Evaluating a classroom deployment of an LLM-based programming assistant that balances student and educator needs. In *Proceedings of the 2024 chi conference on human factors in computing systems*. 1–20.

[18] Sam Lau and Philip Guo. 2023. From "Ban it till we understand it" to "Resistance is futile": How university programming instructors plan to adapt as more students use AI code generation and explanation tools such as ChatGPT and GitHub Copilot. In *Proceedings of the 2023 ACM Conference on International Computing Education Research-Volume 1*. 106–121.

[19] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. 2024. A large-scale survey on the usability of AI programming assistants: Successes and challenges. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. Article 52, 13 pages. https://doi.org/10.1145/3597503.3608128

[20] Q.Vera Liao and S. Shyam Sundar. 2022. Designing for responsible trust in AI systems: A communication perspective. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*. 1257–1268. https://doi.org/10.1145/3531146.3533182

[21] Mark Liffiton, Brad E Sheese, Jaromir Savelka, and Paul Denny. 2023. CodeHelp: Using Large Language Models with guardrails for scalable support in programming classes. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research*. 1–11.

[22] Francesca Lucchetti, Zixuan Wu, Arjun Guha, Molly Q Feldman, and Carolyn Jane Anderson. 2025. Substance beats style: why beginning students fail to code with LLMs. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*. 8541–8610.

[23] Microsoft Copilot. 2025. Copilot and AI agents. https://www.microsoft.com/en-us/microsoft-copilot/copilot-101/copilot-ai-agents.

[24] Ismael Villegas Molina, Audria Montalvo, Benjamin Ochoa, Paul Denny, and Leo Porter. 2025. Leveraging LLM tutoring systems for non-native English speakers in introductory CS courses. In *Proceedings of the 2025 American Society for Engineering Education Annual Conference & Exposition*.

[25] Ipek Ozkaya. 2023. Application of Large Language Models to software engineering tasks: Opportunities, risks, and Implications. *IEEE Software* 40, 3 (2023), 4–8. https://doi.org/10.1109/MS.2023.3248401

[26] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do users write more insecure code with AI assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2785–2799. https://doi.org/10.1145/3576915.3623157

[27] Leo Porter and Daniel Zingaro. 2024. *Learn AI-assisted Python programming: with GitHub Copilot and ChatGPT*. Manning Publications.

[28] James Prather, Juho Leinonen, Natalie Kiesler, Jamie Gorson Benario, Sam Lau, Stephen MacNeil, Narges Norouzi, Simone Opel, Vee Pettit, Leo Porter, et al. 2025. Beyond the hype: A comprehensive review of current trends in Generative AI research, teaching practices, and tools. *2024 Working Group Reports on Innovation and Technology in Computer Science Education* (2025), 300–338.

[29] James Prather, Brent N. Reeves, Paul Denny, Brett A. Becker, Juho Leinonen, Andrew Luxton-Reilly, Garrett Powell, James Finnie-Ansley, and Eddie Antonio Santos. 2023. "It's weird that it knows what I want": Usability and interactions with Copilot for novice programmers. *ACM Trans. Comput.-Hum. Interact.* 31, 1, Article 4 (2023), 31 pages. https://doi.org/10.1145/3617367

[30] James Prather, Brent N Reeves, Juho Leinonen, Stephen MacNeil, Arisoa S Randrianasolo, Brett A. Becker, Bailey Kimmel, Jared Wright, and Ben Briggs. 2024. The widening gap: The benefits and harms of Generative AI for novice Programmers. In *Proceedings of the 2024 ACM Conference on International Computing Education Research - Volume 1*. 469–486. https://doi.org/10.1145/3632620.3671116

[31] replit. 2025. replit - Build apps and sites with AI. https://replit.com/.

[32] Ranjan Sapkota, Konstantinos I. Roumeliotis, and Manoj Karkee. 2025. Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic AI. (2025). https://arxiv.org/abs/2505.19443

[33] Advait Sarkar and Ian Drosos. 2025. Vibe coding: Programming through conversation with artificial intelligence. In *Proceedings of the 36th Psychology of Programming Interest Group Workshop*.

[34] Anshul Shah, Anya Chernova, Elena Tomson, Leo Porter, William G. Griswold, and Adalbert Gerald Soosai Raj. 2025. Students' use of GitHub Copilot for working with large code bases. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1*. 1050–1056. https://doi.org/10.1145/3641554.3701800

[35] Inbal Shani. 2023. *Survey reveals AI's impact on the developer experience*. https://github.blog/2023-06-13-survey-reveals-ais-impact-on-the-developer-experience/

[36] Md Istiak Hossain Shihab, Christopher Hundhausen, Ahsun Tariq, Summit Haque, Yunhan Qiao, and Brian Wise Mulanda. 2025. The effects of GitHub Copilot on computing students' programming effectiveness, efficiency, and processes in brownfield coding tasks. In *Proceedings of the 2025 ACM Conference on International Computing Education Research V.1*. 407–420. https://doi.org/10.1145/3702652.3744219

[37] Lev Tankelevitch, Viktor Kewenig, Auste Simkute, Ava Elizabeth Scott, Advait Sarkar, Abigail Sellen, and Sean Rintel. 2024. The metacognitive demands and opportunities of Generative AI. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–24.

[38] Annapurna Vadaparty, Daniel Zingaro, David H Smith IV, Mounika Padala, Christine Alvarado, Jamie Gorson Benario, and Leo Porter. 2024. CS1-LLM: Integrating LLMs into CS1 instruction. In *Proceedings of the 2024 conference on Innovation and Technology in Computer Science Education v. 1*. 297–303.

[39] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools Powered by Large Language Models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems*. Article 332, 7 pages. https://doi.org/10.1145/3491101.3519665

[40] Ruotong Wang, Ruijia Cheng, Denae Ford, and Thomas Zimmermann. 2024. Investigating and designing for trust in AI-powered code generation tools. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*. 1475–1493. https://doi.org/10.1145/3630106.3658984

[41] Bingyang Wei. 2024. Requirements are all you need: From requirements to code with LLMs. In *2024 IEEE 32nd International Requirements Engineering Conference*. 416–422. https://doi.org/10.1109/RE59067.2024.00049

[42] J.D. Zamfirescu-Pereira, Richmond Y. Wong, Bjoern Hartmann, and Qian Yang. 2023. Why Johnny can't prompt: How non-AI experts try (and fail) to design LLM prompts. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. Article 437, 21 pages. https://doi.org/10.1145/3544548.3581388